

WAFT Architecture Investigation

Summary: This architecture investigation started from the README.

Generated: 2026-01-12 00:43 UTC | Ideas Extracted: 32

What Happened

This architecture investigation started from the README.md entry point and traced through:  CLI entry point (main.py)  Core module structure  Component relationships  Design patterns  Data flow patterns.

Key Findings: WAFT is a self-modifying AI SDK with evolutionary architecture Three core pillars: Substrate, Physics (Scint System), Flight Recorder Manager pattern for orchestration Genome pattern for genetic material representation Generator pattern for document generation Complete lineage tracking for scientific research.

Starting from README.md as the entry point, tracing through: CLI entry point (src/waft/main.py) Core module structure Component relationships Design patterns and architecture decisions.

Purpose: Orchestration layer for major subsystems MemoryManager, SubstrateManager, Empirica-Manager, etc. Single responsibility per manager Consistent interface pattern.

 Read and analyze root README.md  Identify main entry points and core modules  Map component relationships and dependencies (in progress)  Document architecture patterns and design decisions  Generate architecture documentation using WAFT tools.

Hook System: processtavernhook() processes command hooks Narrative generation Dice roll mechanics (D&D ability checks) XP rewards and leveling Command classification (criticalsuccess, critical_failure, etc.).

Key Modules: memory.py - MemoryManager (manages pyrite/ structure) substrate.py - SubstrateManager (agent substrate system) empirica.py - EmpiricaManager (epistemic tracking) gamification.py - GamificationManager (D&D mechanics) github.py - GitHubManager (GitHub integration) tavernkeeper/ - Narrative and gamification system keeper.py - TavernKeeper main class narrator.py - Narrative generation grammars.py - Narrative grammars ai_helper.py - AI helper integration.

Key Modules: chatdistiller.py - ChatDistiller (extract ideas from conversations) stylinggenome.py - StylingGenome (document styling as genetic material) twopagegenerator.py - TwoPageGenerator (2-page PDF generation) pdfgenerator.py - PDFGenerator (composable PDF generation) scientificpdfgenerator.py - ScientificPDFGenerator (self-examination) latexgenerator.py - LaTeXGenerator (LaTeX document generation) componentgenerator.py - ComponentPDFGenerator (component-based PDFs) documentevolutionengine.py - DocumentEvolutionEngine scintdetector.py - ScintDetector (reality fracture detection) pdfresearchtool.py - PDFResearchTool (cross-PDF analysis) pdf_metrics.py - PDFMetrics, PDFMetricsCollector.

WAFt: "A Python framework for directed evolution of self-modifying AI agents" Goal: Observe a "God-Head" agent emerge from evolutionary process Mission: Produce data for "The Physics of Artificial Cognition" research.

Scint System (Reality Fracture Detection) = fitness function Scint Gym tests agents on: SYNTAXTEAR: Formatting errors LOGICFRACTURE: Math errors, contradictions SAFETY_VOID: Harmful content, PII leaks HALLUCINATION: Fabricated facts Fitness = Stability (40%) + Efficiency (30%) + Safety (30%) Fitness < 0.5 = DEATH (evolutionary dead end).

```
from .core.memory import MemoryManager # pyrite memory system
from .core.substrate import SubstrateManager # Agent substrate
from .core.empirica import EmpiricaManager # Epistemic tracking
from .core.gamification import GamificationManager # D&D gamification
from .core.github import GitHubManager # GitHub integration
from .core.tavernkeeper import TavernKeeper # Narrative system.
```

Main Commands (direct on app): new - Create new laboratory verify - Verify project structure evolve - Run evolutionary cycle (Coming Soon) sync - Sync dependencies (uv sync) add - Add dependency init - Initialize WAFT structure info - Show project info serve - Web dashboard (FastAPI + SvelteKit) decide - Decision matrix analysis.

Agent System (core/agent/): base.py - BaseAgent class state.py - Agent state management anatomy.py - Agent anatomy/configuration items.py - Agent items/inventory.

Science System (core/science/): taxonomy.py - LineagePoet (scientific naming) labentry.py - LabEntryGenerator notebook.py - Scientific notebook observer.py - Observer system report.py - Scientific reports tampsyche.py - TAM psyche system.

Decision System: decisioncli.py - DecisionCLI decisionmatrix.py - DecisionMatrixCalculator workflow_decision.py - WorkflowDecisionAnalyzer.

Other Core Modules: analyticscli.py - Analytics commands audit.py - Audit system checkout.py - Checkout system continework.py - Continue work system goal.py - Goal management help.py - Help system persistence.py - Persistence layer proceed.py - Proceed system recap.py - Recap generation reflect.py - Reflection system resume.py - Resume system sessionanalytics.py - Session analytics sessionstats.py - Session statistics visualizer.py - Visualizer workflow.py - Workflow system.

Additional Details

Structure: main.py - FastAPI app creation models.py - API models routes/ - API routes decision.py - Decision endpoints empirica.py - Empirica endpoints git.py - Git endpoints gym.py - Gym endpoints state.py - State endpoints work_efforts.py - Work efforts endpoints.

Content/Conversation ↓ ChatDistiller → DistilledChat (ideas extracted) ↓ StylingGenome → Styling configuration ↓ TwoPageGenerator / PDFGenerator / LaTeXGenerator ↓ Output (PDF, LaTeX, HTML) ↓ PDFMetrics / PDFResearchTool (analysis).

Template System: Jinja2 templates for document generation WeasyPrint: HTML → PDF conversion Style Presets: clinical_standard, premium, professional.

uv: Package management Empirica: Epistemic tracking GitHub: Repository management CrewAI: Agent framework (mentioned in main.py) WeasyPrint: PDF generation FastAPI: Web API SvelteKit: Web frontend.

Evolution System: Document generation and evolution Gamification System: D&D mechanics and narrative Memory System: _pyrite/ structure Agent System: Self-modifying agents Scint System: Reality fracture detection.

Biological Lifecycle: Spawn: Create child agent with mutation Eval: Test fitness in Scint Gym Evolve: Hot-swap to better variant.

OODA Loop (Abstract Methods): observe() - Observe current project state decide() - Make decision using decision engine act() - Execute action reflect() - Reflect on outcome and learn.

Methods: createstructure() - Create pyrite directory structure verifystructure() - Verify structure validity getactivefiles() - Get active files getbacklog_files() - Get backlog files.

Content/Conversation ↓ ChatDistiller.distill_text() ↓ DistilledChat (ideas extracted as IdeaGene) ↓ StylingGenome (styling configuration) ↓ Generator (TwoPageGenerator / PDFGenerator / LaTeXGenerator) ↓ Output (PDF, LaTeX, HTML).

ChatDistiller (evolution/chat_distiller.py): Extracts ideas from conversations Creates IdeaGene objects (genetic material) Categories: concept, decision, insight, action, question.

StylingGenome (evolution/styling_genome.py): Document styling as genetic material Genes: FontGene, MarginGene, ColorGene, LayoutGene Genome ID = SHA-256 hash of styling configuration Enables evolution of document design.

Purpose: Document/content generation PDFGenerator, LaTeXGenerator, ComponentPDFGenerator Builder pattern with fluent interface Preset-based configuration.

Purpose: Content extraction and structuring ChatDistiller extracts ideas from conversations Creates structured data (DistilledChat, IdeaGene) Enables downstream processing.

Framework: Typer (CLI framework) Entry Point: main() function → app() (Typer app).

Core Concept: "The Organism" - self-modifying AI agents with biological lifecycle.

This demonstrates "using WAFT to develop WAFT" - documenting architecture with WAFT's own tools!

Content Breakdown

Type	Count
Decisions	5
Insights	1
Actions	4
Concepts	22
Questions	0